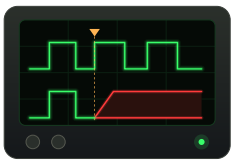


LibreILA (Integrated Logic Analyzer) Datasheet

Version: 0.2
Date: August 2026
Author: Yash Paritkar (github/yashparitkar)

1 Features

- Open-source and free to use: github.com/yashparitkar/libreila
- Probe of any width, described by a CSV file. The same bit order is shared by the trigger vector, the sample buffer and both host drivers
- Pass-through probe splice, combinational, adding no latency to the probed link
- Capture into fabric RAM, one sample per sampling clock edge while armed, read back over AXI4Lite
- Customizable trigger:
  - Condition and mask, one bit per probed bit, reduced by AND or by OR
  - Level, rising edge or falling edge
  - Adjustable trigger position within the captured window
  - Force trigger and disarm from the host, so a condition that never occurs costs a register write rather than a reset
  - Optional external trigger input, selected at synthesis
  - Trigger output for chaining several instances onto one event
- Two independent clock domains, with the arm, disarm and status paths synchronised
- Per-instance identity register, so a system holding several cores can tell them apart
- A serial wrapper is also provided to easily use the ILA with PC, needing no processor and no vendor debug tooling in the design
- A python driver is provided to configure the ILA over the serial wrapper and dump the capture as a VCD for any waveform viewer, with command line and GUI front ends. A baremetal C driver is provided for the bare AXI4Lite core
- Plain VHDL-2008 throughout, with no vendor primitives instantiated
- A default example is provided to use the ILA with a simple 64-bit AXI4Stream interface

2 Application

- RTL debugging
- FPGA prototyping
- Generic logic analyser

3 Description

LibreILA is an in-system Integrated Logic Analyzer (ILA) for FPGA designs, written in plain VHDL. It is spliced into a link inside the fabric and samples it into a circular buffer held in fabric RAM, so nets that never reach a pad are as visible as the ones that do. The host sets a condition on the probed bits, arms the core, and reads back a window of samples from either side of the point at which that condition occurred.

The core observes one flat probe word of `G_PROBE_WIDTH` bits and attaches no meaning to it. That bit order is shared by the trigger, the sample buffer and both drivers, and a build is described entirely by two CSV files.

Two deliverables are provided. `libre_ila` is the core itself, with an AXI4Lite slave port and a baremetal C driver, for a design that already carries a bus master. `libre_ila_uart` wraps it with a UART and a packet parser and is driven from a PC over two wires by the python driver. Neither needs vendor debug tooling in the design.

Parameter	Specification
Probe width	Any width of one bit or more, set at synthesis
Buffer depth	Power of two, two samples or more
Sampling	One sample per `samp_aclk` edge while armed
Storage	`DEPTH` $\times \lceil$ `WIDTH`$/32 \rceil \times 32$ bits of fabric RAM
Probe latency	None, the splice is combinational
Max `samp_aclk`	243 MHz, stock build, PolarFire SoC
Host interface	AXI4Lite slave, or UART 8N1 on the wrapper
Language	VHDL-2008, no vendor primitives

Table 1: Key specifications

Contents

1 Features 1

2 Application 1

3 Description 1

4 Revision History 2

5 Detailed description 3
- 5.1 Overview . . . 3
- 5.2 The probe . . . 4
- 5.3 Capture . . . 5
- 5.4 Trigger . . . 5
- 5.5 External trigger and chaining . . . 6
- 5.6 Clock domains . . . 6
- 5.7 Readout . . . 6
- 5.8 Reset . . . 6
- 5.9 Timing characteristics . . . 7
- 5.10 Timing constraints . . . 7
- 5.11 Device requirements . . . 7

6 Capture procedure 8
- 6.1 Worked example . . . 8

7 Core parameters and interface signals 10
- 7.1 Configuration parameters . . . 10
- 7.2 Portmap . . . 10
- 7.3 Core generation . . . 10

8 Register summary 11
- 8.1 STATUS . . . 13
- 8.2 MGCKEY . . . 14
- 8.3 SAMP_CLK_FREQ . . . 15
- 8.4 WIDTH . . . 16
- 8.5 DEPTH . . . 17
- 8.6 UID . . . 18
- 8.7 SAMP_BUFF_TRIG_IDX . . . 19
- 8.8 SAMP_BUFF_FRST_IDX . . . 20
- 8.9 TRIG_POS . . . 21
- 8.10 ARM_FT . . . 22
- 8.11 TRIG_CFG . . . 23
- 8.12 DISARM . . . 24
- 8.13 TRIG_COND(n) and TRIG_MASK(n) 25
- 8.14 SAMP_BUFF(k,l) . . . 26

9 AXI4Lite UART interface 27
- 9.1 End User Interface . . . 27
- 9.2 Watchdog and error handling . . . 27
- 9.3 UART Packet Format . . . 28

10 Additional references 29

11 Device utilization and performance 30

4 Revision History

Version	Date	Comments
0.0	25 July 2026	Document created.
0.1	4 August 2026	DISARM mapped onto the reserved input register at `0x2C`.
0.2	13 August 2026	Added the capture procedure, reset, timing characteristics, timing constraints and device requirements sections. Corrected the external trigger description: `i_ext_trig` is edge detected but not synchronised inside the core.

Table 2: Revision history

5 Detailed description

This section says how it works: what the blocks are, how a capture proceeds from arming to readout, and where the two clock domains meet. The register-level detail that a driver needs is in Section 8.

5.1 Overview

Everything in the core is arranged around one shared definition of the probe word, `w_probe`. Three blocks sit on top of it:

- **The probe splice.** A pair of mirrored ports shorted combinationally, so the probed link passes straight through and the core reads it in parallel.
- **The trigger comparator.** One condition bit and one mask bit per probed bit, reduced to a single condition and then qualified by the level or edge mode.
- **The sample buffer and the AXI4Lite slave.** A circular buffer in fabric RAM, written every sampling clock while armed and read back as a flat register map.

A capture runs in one direction. The host writes the trigger condition, the mask, the mode and the trigger position, then writes ARM_FT. The core starts sampling into the buffer immediately, overwriting the oldest sample once it wraps, so that at any moment the buffer holds the most recent `G_SAMP_BUFF_DEPTH` samples. When the trigger condition is met the core latches where it happened, keeps sampling for the configured number of post-trigger samples, and stops. DONE then goes high and the buffer holds a window of history either side of the trigger. Sampling has stopped by that point, so the host may read the buffer back at any rate it likes without disturbing the capture. A capture still in progress can be cancelled at any point by writing DISARM, which returns the core to IDLE from either side of the trigger; a capture already finished is left alone.

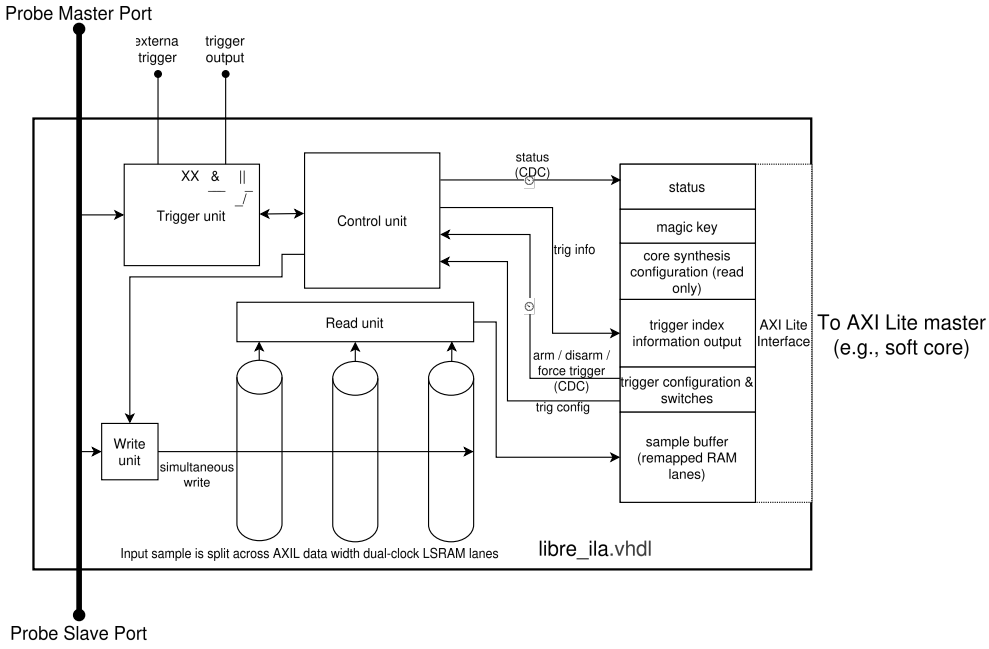


Figure 1: LibreILA core block diagram

The core straddles two clock domains and is deliberate about which side each part lives on. Sampling, triggering and the buffer write port run on `samp_aclk`, the clock of the probed link. The register map and the buffer read port run on `S_AXIL_ACLK`. Only the arm and disarm pulses and the three status bits actually cross, through two-flop synchronisers; the sample data never crosses a synchroniser at all, because the buffer is a dual-port RAM whose read port is simply clocked from the other domain.

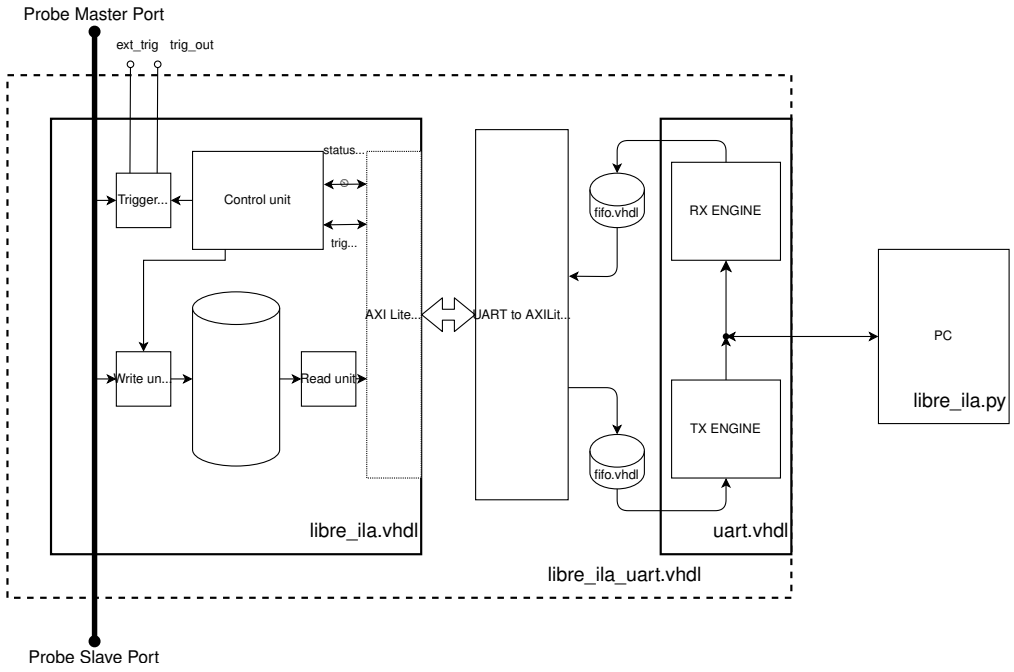


Figure 2: LibreILA UART block diagram

`libre_ila_uart` wraps all of the above with a UART and a packet parser, putting the same register map at the end of a serial link instead of on an AXI bus. Nothing inside the core changes.

5.2 The probe

The core samples one vector, `w_probe`, of `G_PROBE_WIDTH` bits. Signals are packed LSB first in the order they are listed in `portmap.csv`, and that concatenation is the single definition of the probe bit order: the sample buffer, the trigger vector and both drivers inherit it. For the stock 64-bit AXI4Stream build the probe is 67 bits wide, laid out as in Table 3.

Bit range	Signal
63 downto 0	TDATA
64	TLAST
65	TVALID
66	TREADY

Table 3: Probe word of the stock AXI4Stream build

The core is spliced into the link it probes, so the probe appears as a pair of mirrored ports named after the role the core plays on each side, not after the direction the data happens to flow:

- `probe_slave_*` faces the *master* of the probed link, so the core is the slave there, and these ports carry the directions listed in `portmap.csv`.
- `probe_master_*` faces the *slave* of the probed link, so the core is the master there, and every direction is the mirror of its `probe_slave_` twin.

The two sides are shorted combinationally, so the splice is purely pass through and introduces no delay into the probed link. A signal that cannot be spliced in series is tapped in parallel instead, with the `mon` type, which the core reads and never drives.

5.3 Capture

While the ILA is armed it stores one sample of the probe word per rising edge of `samp_aclk` into a circular buffer `G_SAMP_BUFF_DEPTH` samples deep. Once the trigger fires the core keeps capturing for `DEPTH - TRIG_POS - 1` further samples and then stops, so the captured window holds `TRIG_POS` samples of history ahead of the trigger and the rest behind it.

Because the buffer is circular, the oldest sample is not at index zero and the window is almost always split across the wrap point. The core reports where the window starts and where the trigger landed in SAMP_BUFF_FRST_IDX and SAMP_BUFF_TRIG_IDX, and the host unrolls the buffer from there. Figure 3 shows the two views: the capture as it happened, and the same samples as they sit in the buffer.

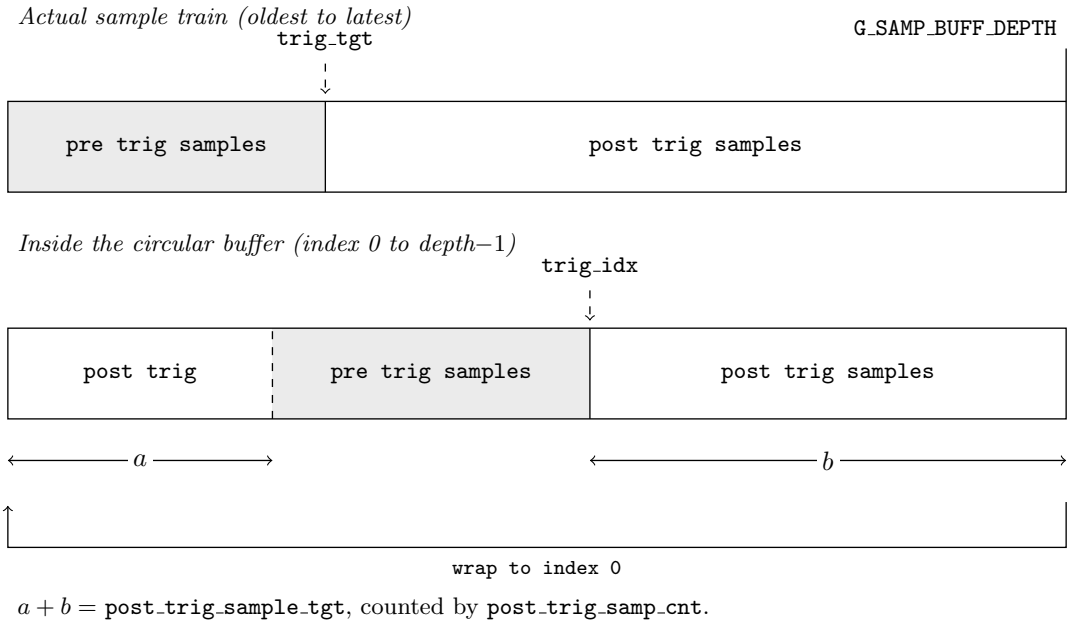


Figure 3: The sample buffer is a circular buffer. The capture starts once the ILA is armed, and the trigger can happen anywhere in the buffer. The host can unroll the buffer in software from the reported start index and trigger index.

That split is also what the post-trigger counter inside the core counts out: it targets $a + b$ samples, not the distance to the end of the buffer.

5.4 Trigger

The trigger is one condition bit per probed bit, held in two registers of identical layout. TRIG_COND carries the value each probe bit has to take to match, TRIG_MASK decides whether that bit takes part at all. The enabled bits are then reduced to a single condition, by AND or by OR, and that single condition is what the level and edge modes act on (Table 4).

EDGE	FALLING	Mode	Triggers when the condition
0	x	Level	*is* true
1	0	Rising	*becomes* true
1	1	Falling	*becomes* false

Table 4: Trigger modes, `TRIG_CFG` bits 2 and 1

The distinction matters at arming time. In level mode a condition that already holds when the ILA is armed fires on the very first sample and captures nothing of interest, as “trigger when TVALID is high”

would on a mostly idle stream. The edge modes wait for a transition instead, and a condition that is already true at the moment of arming is not mistaken for a rising edge.

Edge detection is applied to the reduced condition and not per probe bit. Asking for "bit 5 rises *and* bit 9 falls in the same sample" is therefore not expressible.

Note that an all-zero TRIG_MASK in AND mode matches vacuously and fires on the first sample after arming. Use OR mode with an all-zero mask if the intent is never to trigger on the probe.

5.5 External trigger and chaining

Setting `G_EXTERNAL_TRIG` to 1 replaces the trigger comparator with the `i_ext_trig` pin: in that build the ILA triggers only on the rising edge of that pin, and the three TRIG_CFG mode bits, TRIG_COND and TRIG_MASK are all ignored. The trigger vector logic is not synthesised at all, which is why the external trigger build is the smaller of the two in Table 14.

The edge is detected by a single flop in the sampling domain and `i_ext_trig` is **not** synchronised inside the core. It must therefore be driven from logic already running on `samp_aclk`, or synchronised externally before it reaches the pin.

The resulting trigger is driven out on `o_trig_out` in every build, so several instances can be chained to capture one event across different parts of a design: `o_trig_out` of one instance drives `i_ext_trig` of the next. Chained instances must share a sampling clock, or carry a synchroniser between them, for the same reason.

5.6 Clock domains

The capture side runs entirely in the sampling clock domain, driven by `samp_aclk`, which should be fed the clock the probed link runs on. The AXI4Lite port is a separate domain on `s_axil_aclk`. The arm and disarm pulses and the ARMED, TRIGD and DONE status bits cross between them through two-flop synchronisers and so lag by a couple of cycles; the internal STATE field of the status register is not synchronised and is there for debugging only.

Arm and disarm cross on separate paths, each a write-toggled bit resynchronised and edge-detected on the sampling side. Two writes close together can therefore land in either order in the sampling domain. A host that writes both back to back should leave at least five sampling clock periods between them if it cares which one wins.

Readout needs no synchroniser at all. The sample buffer is a simple dual-port RAM whose write port is clocked by `samp_aclk` and whose read port is clocked by `s_axil_aclk`, so the sample data never crosses a synchroniser and never needs to.

5.7 Readout

Samples are stored one 32-bit lane per slice of the probe word, so lane n carries probe bits $32n + 31$ downto $32n$ and the last lane is zero padded in its upper bits. On readout the lanes are padded further, up to `C_AXIL_STRIDE` registers per sample. The trigger vector uses that same padded layout, so a captured sample and a trigger condition are directly comparable word for word, and a sample read back can be written straight into TRIG_COND to trigger on its recurrence.

5.8 Reset

The core takes two resets, one per clock domain. `i_rst_sync` is active high and is sampled in the sampling clock domain; it clears the capture state machine, the write pointer and the trigger index. `s_axil_aresetn` is active low and is sampled in the AXI4Lite domain; it clears the register map and the AXI4Lite state machines. The sample buffer contents are not cleared by either, since a capture always overwrites the whole buffer before it can complete.

Both resets must be held for at least two periods of the clock that samples them. `libre_ila_uart` drives `s_axil_aresetn` internally from `i_rst_sync`, so the wrapper takes a single reset input; hold it for at least two periods of the *slower* of the two clocks.

5.9 Timing characteristics

The figures in Table 5 are cycle counts through the core's own logic and hold for any build. The one cycle of uncertainty on the arm and disarm paths is the resolution time of the first synchroniser flop.

Parameter	Value
Probe input to probe output, through the splice	Combinational, 0
ARM_FT write accepted to first sample stored	4 to 5 `samp_aclk`
DISARM write accepted to IDLE	4 to 5 `samp_aclk`
Trigger condition true to the sample recorded at SAMP_BUFF_TRIG_IDX	0, it is that sample
Trigger to DONE	`DEPTH`−`TRIG_POS`−1 `samp_aclk`
Capture state change to ARMED, TRIGD or DONE readable over AXI4Lite	2 `s_axil_aclk`
Minimum spacing between an ARM_FT and a DISARM write for the order to be guaranteed	5 `samp_aclk`

Table 5: Timing characteristics

The trigger index is exact: the sample stored at SAMP_BUFF_TRIG_IDX is the one on which the condition held, in level mode, or the one on which it changed, in the edge modes. No correction for pipeline delay is needed on the host side.

Readout time over the UART wrapper follows from the packet format of Section 9.3. A capture of `DEPTH` samples is $a \times$`DEPTH` words, sent in packets of at most 127 words, each carrying a six byte header in both directions. For the stock build that is 8192 words in 65 packets, about 33.5 kB, which at 115 200 baud 8N1 takes roughly 2.9 seconds.

5.10 Timing constraints

`samp_aclk` and `s_axil_aclk` are unrelated. Declare them as asynchronous to one another in the constraint file so that the tool does not attempt to close timing between the domains; in SDC that is `set_clock_groups -asynchronous`. The paths that genuinely cross are the arm and disarm toggles and the three status bits, all of which carry their own two-flop synchronisers, and the sample buffer, which crosses inside the RAM and needs no constraint.

Where the two clocks are driven from the same source, or where the design constrains them together for other reasons, a maximum delay on the crossing registers is the narrower alternative.

5.11 Device requirements

The core is plain VHDL-2008 and uses only `ieee.std_logic_1164`, `ieee.numeric_std` and `ieee.math_real`. No vendor primitive is instantiated anywhere, and no vendor library is referenced, so the sources compile as they stand under any VHDL-2008 tool. They are simulated with GHDL in the supplied test suite.

The one thing the core asks of a device is that its synthesis tool infer a **simple dual-port RAM with independent read and write clocks** from the sample buffer, which is written on `samp_aclk` and read on `s_axil_aclk`. Every current FPGA family has such a block. Where a tool declines to infer it the design still builds and still behaves correctly, but the buffer falls back to registers and the resource cost rises sharply, so it is worth checking the synthesis report on a new family.

The buffer carries a `syn_ramstyle` attribute of `"lsram"`, which Synplify and Libero honour and other tools ignore. It is the only vendor-specific text in the sources and changing or removing it does not affect behaviour.

The figures quoted throughout this document were measured on a PolarFire SoC (MPFS Discovery Kit).

6 Capture procedure

A capture is a short sequence of register accesses followed by a bulk read. The steps below give the general form; the offsets quoted are those of the stock build, where the stride a is 4. Table 10 gives the offsets symbolically for any build.

1. **Confirm the core.** Read MGCKEY at `0x04` and check that it reads `0xB01DFACE`. Read UID at `0x14` if the system holds more than one ILA.
2. **Read the geometry.** WIDTH at `0x0C` and DEPTH at `0x10` give the probe width and the buffer depth, from which `C_N_LANES` and the stride a follow. SAMP_CLK_FREQ at `0x08` gives the time axis. Every remaining offset is derived from these, so nothing needs to be known about the build in advance.
3. **Set the trigger position.** Write TRIG_POS at `0x20` with the number of samples of history to keep before the trigger. The core will capture `DEPTH`$-$`TRIG_POS`-1 samples after it.
4. **Set the condition.** Write TRIG_COND$(0 \ldots a-1)$ with the value each probe bit must take, and TRIG_MASK$(0 \ldots a-1)$ with a 1 for each bit that takes part. Bits left masked off are don't cares.
5. **Set the mode.** Write TRIG_CFG at `0x28` to select AND or OR reduction and level, rising or falling sensitivity.
6. **Arm.** Write any value to ARM_FT at `0x24`. Sampling begins within five sampling clocks.
7. **Wait.** Poll STATUS at `0x00` until DONE, bit 2, is set. A second write to ARM_FT forces the trigger if the condition turns out never to occur; a write to DISARM at `0x2C` cancels the capture and returns the core to IDLE.
8. **Locate the window.** Read SAMP_BUFF_FRST_IDX at `0x1C` and SAMP_BUFF_TRIG_IDX at `0x18`. Both are valid once DONE is set.
9. **Read the buffer.** Read $a \times$`DEPTH` words from `0x50`. Sample k, lane l sits at `0x50` $+ 4(ak + l)$.
10. **Unroll.** Sample i of the window, counting from the oldest, is at buffer index (FRST_IDX$+i$) mod `DEPTH`. The trigger sample sits at position (TRIG_IDX $-$ FRST_IDX) mod `DEPTH` within the unrolled window.

Both supplied drivers carry out this sequence. The python driver additionally unrolls the buffer, applies the signal names from `portmap.csv` and writes the result as a VCD file.

6.1 Worked example

Using the stock 64-bit AXI4Stream build of Table 3, capture the start of a packet: trigger when TVALID and TLAST are both high, on the rising edge of that condition, keeping 512 samples of history.

TLAST is probe bit 64 and TVALID is probe bit 65, so both fall in trigger word 2, which carries probe bits 95 downto 64, at bit positions 0 and 1 respectively. TDATA and TREADY take no part and their mask bits stay at zero, as do the padding bits above the probe width. Table 6 gives the resulting writes.

Offset	Register	Value	Meaning
`0x20`	TRIG_POS	`0x00000200`	512 pre-trigger samples, so 1535 after the trigger
`0x38`	TRIG_COND(2)	`0x00000003`	TLAST and TVALID must both be 1
`0x48`	TRIG_MASK(2)	`0x00000003`	only those two bits take part
`0x28`	TRIG_CFG	`0x00000002`	AND reduction, edge mode, rising
`0x24`	ARM_FT	any	arms the core

Table 6: Register writes for the worked example. TRIG_COND and TRIG_MASK words 0, 1 and 3, at `0x30`, `0x34`, `0x3C`, `0x40`, `0x44` and `0x4C`, are left at zero.

Poll STATUS at `0x00` until bit 2 is set, then read SAMP_BUFF_FRST_IDX and SAMP_BUFF_TRIG_IDX, then read 8192 words from `0x50`. Sample k occupies the four words at `0x50` $+ 16k$, of which the first three carry probe bits 95 downto 0 and the fourth is padding. Over the UART wrapper the readout is 65 packets and takes about 2.9 seconds at 115 200 baud.

Had the same condition been set in level mode, with TRIG_CFG at `0x00000000`, a stream already sitting with TVALID and TLAST high would have triggered on the very first sample and captured nothing of

interest. The edge modes exist for exactly that case.

7 Core parameters and interface signals

Everything in this section applies to both `libre_ila` and `libre_ila_uart`; the wrapper adds the generics and ports marked as such.

Three quantities are derived from the generics and are referred to throughout the rest of this document:

- `C_N_LANES` $= \lceil$`G_PROBE_WIDTH`$/32 \rceil$, the number of 32-bit lanes one sample occupies in the RAM.
- `C_AXIL_STRIDE` (a), `C_N_LANES` rounded up to the next power of two with a minimum of 4. This is the number of registers one sample, and one trigger vector, occupies on the bus.
- `C_ADDR_WIDTH` $= \log_2($`G_SAMP_BUFF_DEPTH`$)$, the width of a sample index.

For the stock 64-bit AXI4Stream build `G_PROBE_WIDTH` is 67, so `C_N_LANES` is 3, a is 4 and `C_ADDR_WIDTH` is 11.

7.1 Configuration parameters

The core is parameterised by a set of generics, which are listed in Table 7. The wrapper adds the UART parameters, which are also listed in the same table. The AXI4Lite parameters are fixed and should not be changed. The parameters G_PROBE_WIDTH is generated from the portmap.csv file and must be generated rather than changed by hand.

7.2 Portmap

The probe ports are not fixed. They are written by the generator from `portmap.csv`, one line per probed signal as `signal,width,type`, listed LSB first. The same file gives the python driver the names it puts in the VCD, so one file describes the probe for the hardware and for the host at once.

`type` is the direction the signal has on the `probe_slave_` side, that is the direction the probed link's slave would give it (Table 8). There is no `inout`: a signal assignment cannot pass a bidirectional line through, and an ILA belongs on the fabric side of the IO buffer where the bus is not bidirectional yet. Where a link genuinely cannot be spliced, tap it with `mon` instead.

The remaining ports are fixed and are listed in Table 9. The AXI4Lite slave port carries the standard AW, W, B, AR and R channel signals with the `s_axil_` prefix and is not enumerated here.

7.3 Core generation

A build is described entirely by two CSV files in `codegen/`: `portmap.csv` for the probe, and `configuration.csv` for the generic values, one line per generic as `generic,type,value`. Running the generator emits a complete set of VHDL sources for that configuration:

```
python3 codegen/code_generator.py \
        --portmap codegen/portmap.csv \
        --config  codegen/configuration.csv
```

Left at their shipped defaults, the two files describe the stock 64-bit AXI4Stream build. `GEN_TYPE` in the configuration selects what comes out: 0 emits the bare AXI4Lite core alone, 1 emits it together with the UART wrapper and the FIFO and UART blocks it instantiates. Add the emitted files to the project, instantiate the core or the wrapper, and splice the probe ports into the link to be observed. Section 6 covers driving it once it is in. The CSV formats are documented in full in `codegen/README.md` in the repository.

8 Register summary

The core presents a flat map of 32-bit registers on its AXI4Lite slave port; a register's byte address is its index times four. The map is laid out in three blocks: the **output block** first, then the **input block**, then the **sample buffer**.

The output block is eight registers in every build and sits at a fixed place; the input block and the sample buffer both move with the probe width. A host therefore reads the geometry of the core out of the output block at start-up and derives the rest of the map from it. Both supplied drivers work this way, so re-synthesising with a different probe width or buffer depth needs no driver rebuild.

Three kinds of register appear in the map:

- **Output, read only.** Status, and the synthesis-time geometry of the core.
- **Input, read/write.** The trigger configuration, plus the write-only arm register.
- **Sample buffer, read only.** The capture itself, a registers per sample.

Table 10 lists the whole map. Offsets in the input block and above are given symbolically in a, with the value for the stock build ($a = 4$) in parentheses.

The AXI4Lite decoder *truncates* an address it cannot decode rather than rejecting it, so a misaligned or out of range access silently aliases onto a real register. Both supplied drivers check alignment on the host side and the UART wrapper checks it in hardware, because two registers of the input block act on the write rather than on the data: a stray write to ARM_FT would arm the core, and one to DISARM would throw away a capture it was part way through.

Offset	Name	Access	Reset	Description
Output block				
0x00	STATUS	R	0x00000000	ILA status and state
0x04	MGCKEY	R	0xB01DFACE	Magic key, identifies the core type
0x08	SAMP_CLK_FREQ	R	G_SAMP_CLK_FREQ	Sampling clock frequency in Hz
0x0C	WIDTH	R	G_PROBE_WIDTH	Total probed bits
0x10	DEPTH	R	G_SAMP_BUFF_DEPTH	Sample buffer depth in samples
0x14	UID	R	G_UID	Identity of this instance
0x18	SAMP_BUFF_TRIG_IDX	R	0x00000000	Buffer index of the trigger sample
0x1C	SAMP_BUFF_FRST_IDX	R	0x00000000	Buffer index of the oldest sample
Input block				
0x20	TRIG_POS	R/W	0x00000000	Number of pre-trigger samples
0x24	ARM_FT	W	0x00000000	Any write arms the ILA, or forces the trigger if it is already armed
0x28	TRIG_CFG	R/W	0x00000000	Trigger mode: AND/OR, level/edge, rising/falling
0x2C	DISARM	W	0x00000000	Any write cancels a capture in progress and returns the ILA to IDLE. Ignored once the capture is DONE
0x30 + 4n	TRIG_COND(n)	R/W	0x00000000	Trigger condition, word n, $n = 0 \ldots a-1$ (0x30–0x3C)
0x30 + 4n + 4a	TRIG_MASK(n)	R/W	0x00000000	Trigger mask, word n, $n = 0 \ldots a-1$ (0x40–0x4C)
Sample buffer				
0x30 + 8a + 4($ak + l$)	SAMP_BUFF(k,l)	R	—	Sample k, lane l, for $k = 0 \ldots$ DEPTH-1 and $l = 0 \ldots a-1$ (base 0x50)

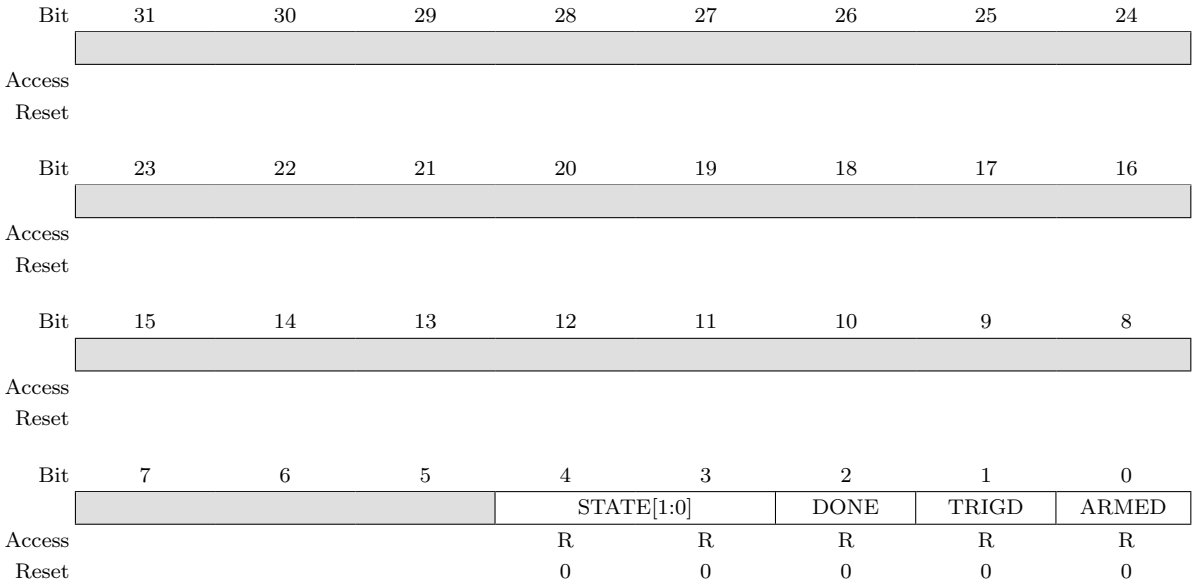
Table 10: Register map. Offsets are from the AXI4Lite base address.

The map therefore holds $8 + 4 + 2a$ control registers followed by $a \times$G_SAMP_BUFF_DEPTH buffer registers. For the stock build that is 20 control registers and 8192 buffer registers, ending at byte offset 0x8050.

8.1 STATUS

Name: STATUS
Register Set: Output block
Offset: 0x00
Reset: 0x00000000
Property: Read-only

Status register. It reports the current state of the ILA core. The ARMED, TRIGD and DONE bits are synchronised into the AXI4Lite domain with two flops and so lag the capture domain slightly. STATE is the raw internal state, is not synchronised, and is provided for debugging.



Bits 4:3 – STATE[1:0] Internal state of the capture state machine, not synchronised to the AXI4Lite domain.

Value	Description
0b00	IDLE, waiting to be armed
0b01	ARMED, capturing, trigger not yet seen
0b10	TRIGD, trigger seen, capturing post-trigger samples
0b11	DONE, capture complete, buffer readable

Bit 2 – DONE Set when the capture has completed and the sample buffer holds a full window.
Bit 1 – TRIGD Set when the trigger has fired and the core is capturing its post-trigger samples.
Bit 0 – ARMED Set while the core is armed and capturing.

8.2 MGCKEY

Name: MGCKEY
Register Set: Output block
Offset: `0x04`
Reset: `0xB01DFACE`
Property: Read-only

Magic key register. It reads back a fixed constant, and is the cheapest way for a host to confirm that it is talking to a LibreILA core at the base address it was given, before it trusts anything else in the map.

The constant is the same in every build: it identifies the core *type*, never the individual core. Telling one instance from another is the job of UID at `0x14`.

Bit	31	30	29	28	27	26	25	24
	MGCKEY[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	1	0	1	1	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	MGCKEY[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	1	1	1	0	1

Bit	15	14	13	12	11	10	9	8
	MGCKEY[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	1	1	1	1	1	0	1	0

Bit	7	6	5	4	3	2	1	0
	MGCKEY[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	1	1	0	0	1	1	1	0

Bits 31:0 – MGCKEY[31:0] Constant `0xB01DFACE`.

8.3 SAMP_CLK_FREQ

Name: SAMP_CLK_FREQ
Register Set: Output block
Offset: `0x08`
Reset: `G_SAMP_CLK_FREQ`
Property: Read-only

Sampling clock frequency in Hz, as given by `G_SAMP_CLK_FREQ` at synthesis. Nothing inside the core uses it; it is here so that a host can turn a sample index into a time and label the waveform axis without being told the clock rate separately.

Bit	31	30	29	28	27	26	25	24
	SAMP_CLK_FREQ[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	SAMP_CLK_FREQ[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	SAMP_CLK_FREQ[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	SAMP_CLK_FREQ[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – SAMP_CLK_FREQ[31:0] Frequency of `samp_aclk` in Hz. Here and below, a reset of `x` means the bit is fixed at synthesis by the corresponding generic.

8.4 WIDTH

Name: WIDTH
Register Set: Output block
Offset: `0x0C`
Reset: `G_PROBE_WIDTH`
Property: Read-only

Total number of probed bits. Together with DEPTH this is everything a host needs in order to derive the rest of the register map, since the lane count and the stride a both follow from it.

Bit	31	30	29	28	27	26	25	24
	WIDTH[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	WIDTH[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	WIDTH[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	WIDTH[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – WIDTH[31:0] `G_PROBE_WIDTH`, the width of the probe word in bits.

8.5 DEPTH

Name: DEPTH
Register Set: Output block
Offset: `0x10`
Reset: `G_SAMP_BUFF_DEPTH`
Property: Read-only

Depth of the sample buffer in samples. Always a power of two, so a host can take its logarithm to get the width of a sample index.

Bit	31	30	29	28	27	26	25	24
	DEPTH[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	DEPTH[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	DEPTH[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	DEPTH[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – DEPTH[31:0] `G_SAMP_BUFF_DEPTH`, the number of samples the buffer holds.

8.6 UID

Name: UID
Register Set: Output block
Offset: `0x14`
Reset: `G_UID`
Property: Read-only

Identity of this particular core, fixed at synthesis by `G_UID`. Nothing inside the core reads it and no part of the register map moves with it; it exists so that a system carrying several ILAs can be told which one it has reached. Where MGCKEY answers *is this a LibreILA*, UID answers *which LibreILA*.

A host walking a list of base addresses, or of serial ports, checks MGCKEY to know a core is there at all and then reads UID to find out which one it is. Every value is legal and none is validated, by the hardware or by either driver. Zero is what a core built without a `G_UID` reports and means no more than *unset*.

Bit	31	30	29	28	27	26	25	24
	UID[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	UID[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	UID[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	UID[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

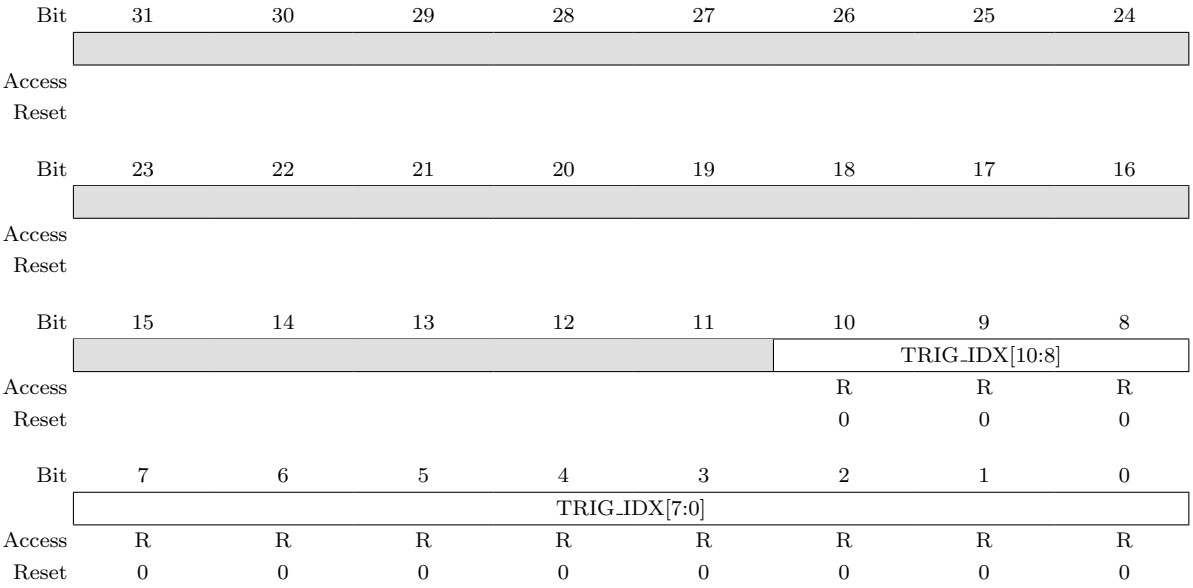
Bits 31:0 − UID[31:0] `G_UID`, the identity this instance was given at synthesis. Bit 31 always reads zero: the generic is a `natural`, whose upper bound is `0x7FFFFFFF`.

8.7 SAMP_BUFF_TRIG_IDX

Name: SAMP_BUFF_TRIG_IDX
Register Set: Output block
Offset: `0x18`
Reset: `0x00000000`
Property: Read-only

Index, within the circular sample buffer, of the sample on which the trigger fired. Read it once DONE is set. Note that this is where the trigger *landed* in the buffer, which is not the same thing as TRIG_POS: TRIG_POS is the number of pre-trigger samples the host asked for, this is the raw buffer index needed to go and find them.

The field is `C_ADDR_WIDTH` bits wide, and the diagram below shows the stock 2048-deep build where that is 11 bits. Bits above `C_ADDR_WIDTH` always read zero.

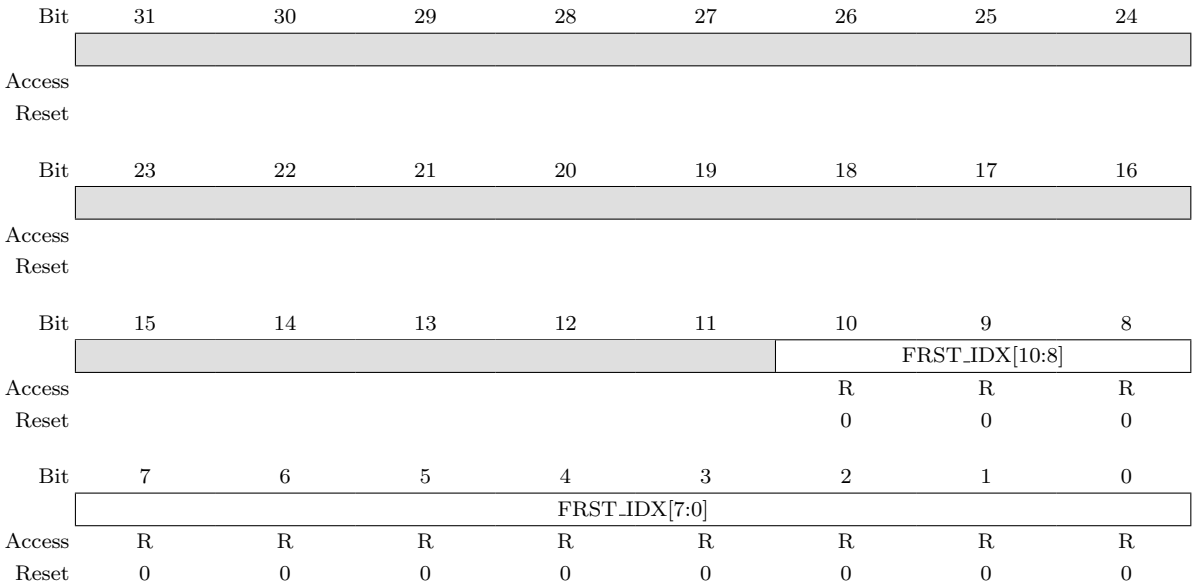


Bits 10:0 – TRIG_IDX Buffer index of the trigger sample, valid once DONE is set.

8.8 SAMP_BUFF_FRST_IDX

Name: SAMP_BUFF_FRST_IDX
Register Set: Output block
Offset: 0x1C
Reset: 0x00000000
Property: Read-only

Index, within the circular sample buffer, of the oldest sample of the captured window. Because the buffer wraps, this is where a host has to start reading in order to get the capture in time order: sample i of the unrolled window lives at buffer index (FRST_IDX + i) mod DEPTH.



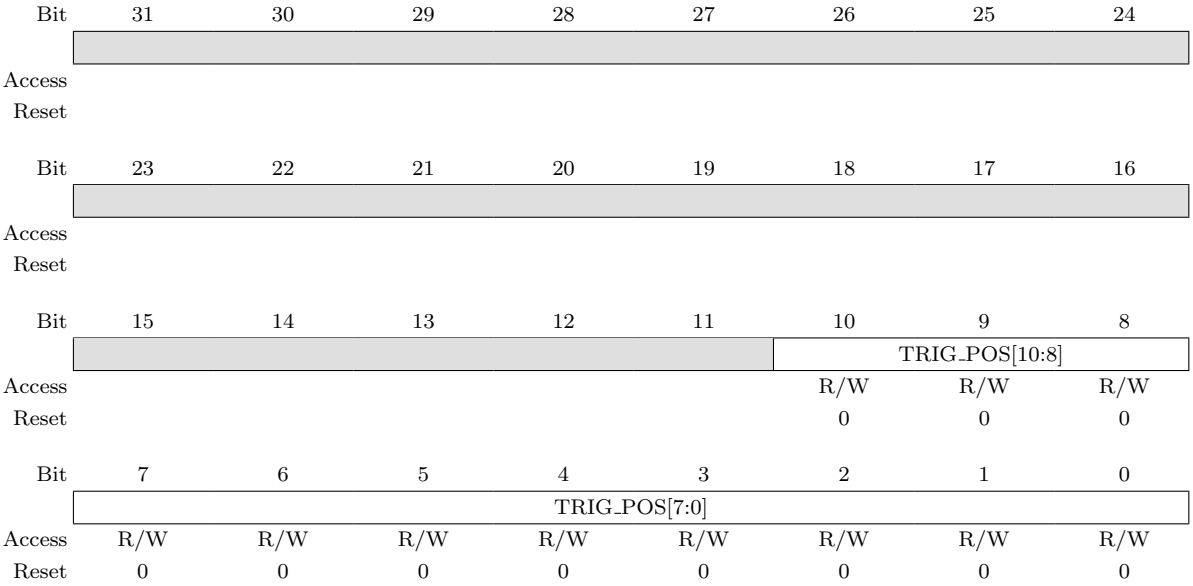
Bits 10:0 – FRST_IDX Buffer index of the oldest sample, valid once DONE is set.

8.9 TRIG_POS

Name: TRIG_POS
Register Set: Input block
Offset: 0x20
Reset: 0x00000000
Property: Read/Write

Trigger position, in samples. It is the number of samples of history kept ahead of the trigger, so the core captures for `DEPTH - TRIG_POS - 1` samples after the trigger fires before going to DONE. Zero puts the trigger at the very start of the window, `DEPTH`−1 puts it at the end.

Write this before arming. The field is `C_ADDR_WIDTH` bits wide; the diagram shows the stock 2048-deep build.



Bits 10:0 – TRIG_POS Number of pre-trigger samples.

8.10 ARM_FT

Name: ARM_FT
Register Set: Input block
Offset: 0x24
Reset: 0x00000000
Property: Write-only

Arm and force-trigger register. **Any** write to this register acts; the value written is ignored entirely. A write while the core is IDLE or DONE arms it and starts the capture, and a write while it is already armed forces the trigger immediately, which is how a capture is recovered when the configured condition never occurs.

Configure TRIG_POS, TRIG_CFG, TRIG_COND and TRIG_MASK before writing here.

Bit	31	30	29	28	27	26	25	24
	ARM_FT[31:24]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	ARM_FT[23:16]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	ARM_FT[15:8]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	ARM_FT[7:0]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

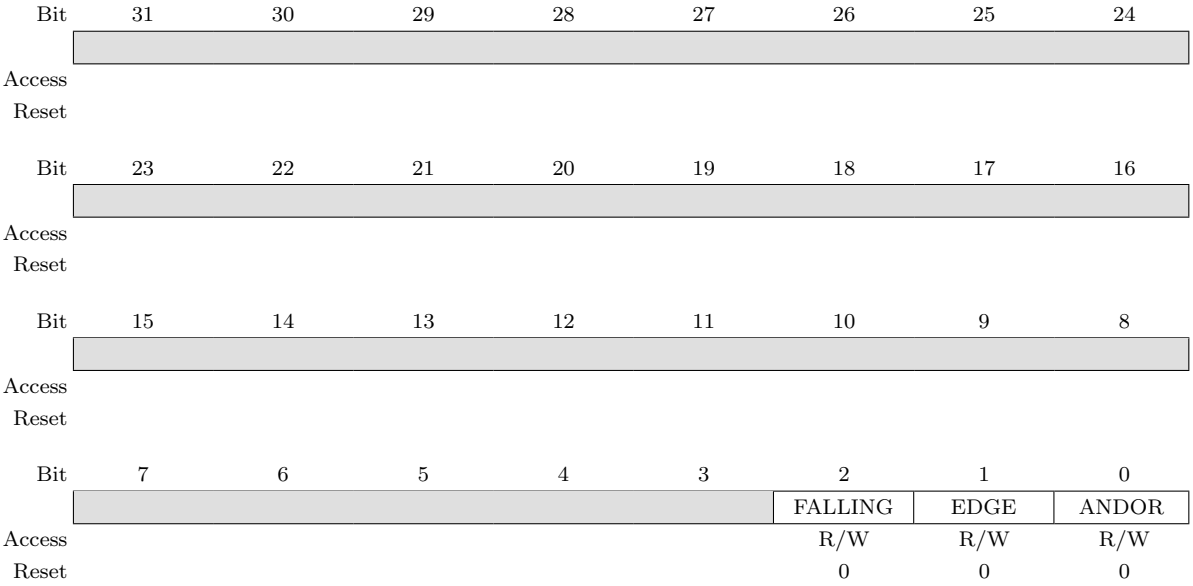
Bits 31:0 – ARM_FT[31:0] The write itself is the command, the data is a don't care.

8.11 TRIG_CFG

Name: TRIG_CFG
Register Set: Input block
Offset: 0x28
Reset: 0x00000000
Property: Read/Write

Trigger configuration. ANDOR chooses how the mask-enabled bits are reduced to a single condition, and EDGE and FALLING then decide what counts as a trigger on that condition. The reset value of zero gives an AND reduction in level mode.

These bits are ignored when the core is built with `G_EXTERNAL_TRIG` set to 1, where the trigger is always the rising edge of `i_ext_trig`.



Bit 2 – FALLING Edge polarity. Only read when EDGE is 1.

Value	Description
0b0	Trigger on the rising edge of the condition
0b1	Trigger on the falling edge of the condition

Bit 1 – EDGE Level or edge sensitivity.

Value	Description
0b0	Trigger while the condition is true
0b1	Trigger on a transition of the condition

Bit 0 – ANDOR Reduction of the mask-enabled bits.

Value	Description
0b0	AND, the condition holds once every enabled bit matches
0b1	OR, the condition holds as soon as any enabled bit matches

8.12 DISARM

Name: DISARM
Register Set: Input block
Offset: 0x2C
Reset: 0x00000000
Property: Write-only

Disarm register. **Any** write to this register acts; the value written is ignored entirely, the same way ARM_FT works. A write while the core is ARMED or TRIGD cancels the capture and returns it to IDLE, so a capture waiting on a condition that never comes, or one started by the wrong condition, can be taken back without resetting the core.

Cancelling from TRIGD abandons the post-trigger samples still owed. The samples already taken stay in the buffer, but SAMP_BUFF_TRIG_IDX goes back to zero with the state, so nothing that is read out afterwards claims to be a capture.

A write while the core is IDLE does nothing, and a write while it is DONE is *ignored*. Disarm cancels a capture in progress, it does not discard a finished one; arm again to start the next.

A trigger arriving in the same cycle as a disarm wins, so a race between the two leaves the core in TRIGD rather than IDLE. A second disarm then cancels it.

Bit	31	30	29	28	27	26	25	24
	DISARM[31:24]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	DISARM[23:16]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	DISARM[15:8]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	DISARM[7:0]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – DISARM[31:0] The write itself is the command, the data is a don't care.

8.13 TRIG_COND(n) and TRIG_MASK(n)

Name: TRIG_COND(n) / TRIG_MASK(n)
Register Set: Input block
Offset: `0x30` + $4n$ / `0x30` + $4a + 4n$
Reset: `0x00000000`
Property: Read/Write

The trigger vector, one bit per probed bit, spread over a registers each. Word n carries probe bits $32n+31$ downto $32n$, which is exactly the layout one sample of the buffer uses, so a captured sample can be written straight back as a condition.

For probe bit i, the TRIG_COND bit is the value that bit has to take in order to match, and the TRIG_MASK bit decides whether it takes part at all, 1 enabling it and 0 making it a don't care. Bits above `G_PROBE_WIDTH` up to the stride boundary are padding; leave their mask bits at 0.

Bit	31	30	29	28	27	26	25	24
	TRIG_COND(n)[31:24] / TRIG_MASK(n)[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	TRIG_COND(n)[23:16] / TRIG_MASK(n)[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	TRIG_COND(n)[15:8] / TRIG_MASK(n)[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	TRIG_COND(n)[7:0] / TRIG_MASK(n)[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 Trigger condition and mask for probe bits $32n + 31$ downto $32n$.

8.14 SAMP_BUFF(k,l)

Name: SAMP_BUFF(k,l)
Register Set: Sample buffer
Offset: `0x30` + $8a + 4(ak + l)$
Reset: —
Property: Read-only

The capture itself. Sample k occupies a consecutive registers, of which the first `C_N_LANES` carry the probe word, lane l holding probe bits $32l + 31$ downto $32l$. The last lane is zero padded in its upper bits, and the registers between `C_N_LANES` and the stride boundary are padding that reads back as zero.

The padding costs bus address space and not RAM: the buffer itself is only `C_N_LANES` lanes wide.

Read the buffer only once DONE is set, and start from SAMP_BUFF_FRST_IDX to get the window in time order.

Bit	31	30	29	28	27	26	25	24
	SAMP_BUFF(k,l)[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	SAMP_BUFF(k,l)[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	SAMP_BUFF(k,l)[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	SAMP_BUFF(k,l)[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 Probe bits $32l + 31$ downto $32l$ of sample k. The contents are undefined until a capture has completed.

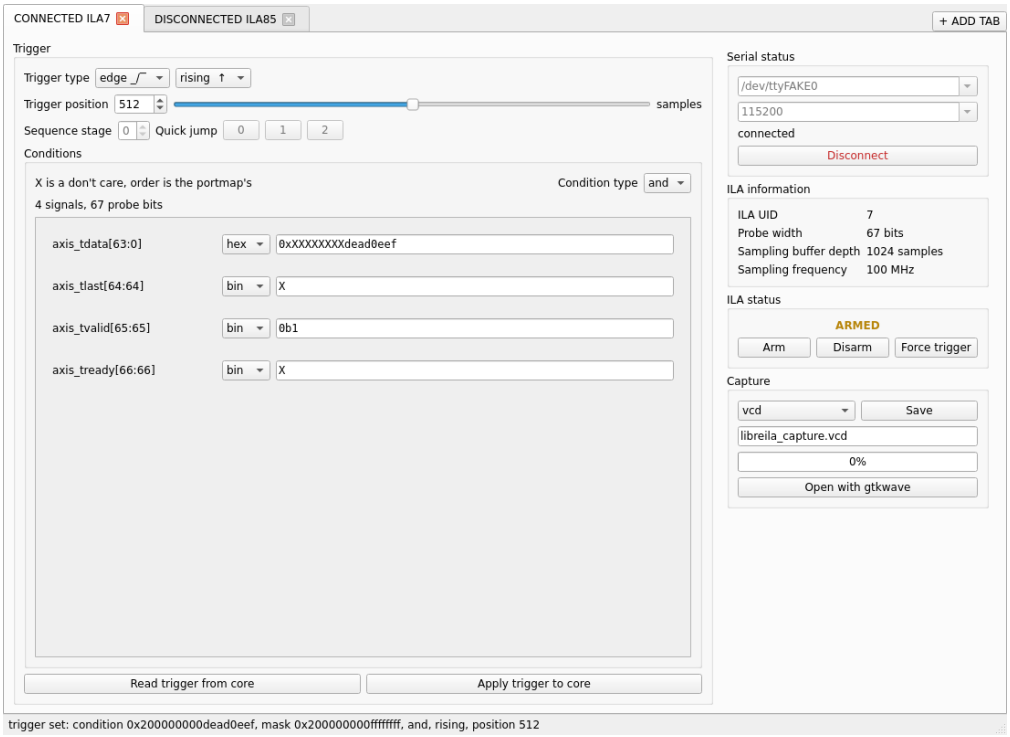


Figure 4: Image of the GUI showing ILA in armed state.

9 AXI4Lite UART interface

`libre_ila_uart` wraps the core with a UART and a small packet parser, so that a design with no processor, or with no spare AXI master, can still be debugged from a PC over a two-wire serial link. The wrapper instantiates the core and carries all of its generics, adding the FIFO depths and the baud rate. The register map seen from the PC is exactly the map of Table 10, reached by address rather than through any command protocol.

The supplied python driver speaks the packet format, splits long transfers, unrolls the circular buffer and writes a VCD with the signals named from `portmap.csv`, which any waveform viewer will open.

9.1 End User Interface

The UART wrapper can be interfaced with any USBTTL adapter, and the supplied python driver provides a both command line and GUI interface. The GUI is shown in Figure 4 with the ILA in the armed state. Both drivers are available from the repository, with their usage documented in `drivers/python/README.md` and `drivers/baremetal/README.md`.

9.2 Watchdog and error handling

Two safety nets keep a bad link from wedging the wrapper.

A **watchdog** counts while the wrapper is in any non-idle state and is cleared whenever a byte makes progress. After `G_AXIL_CLK_FREQ` counts, that is one second without progress, the wrapper resets to idle and abandons whatever it had parsed. A timeout therefore loses the whole transaction rather than half applying it, and the host recovers by restarting from its sync byte; since the idle state discards every byte that is not a sync byte, the link resynchronises on its own.

Each request is also **sanity checked** once its base address is in. The address must be four byte aligned, because the core's decoder truncates rather than rejects and a misaligned write could land on ARM_FT. The receive FIFO is checked for overflow, since the UART writes into it unconditionally. A request that fails

either check is answered but never put on the AXI bus: a read returns the promised number of zero words, and a write still has its payload drained, so the link stays in step.

9.3 UART Packet Format

The PC is the master of the link. Every transaction is one request packet from the PC and one response packet from the wrapper, both beginning with a sync byte and a six byte header.

Field	Size	PC to wrapper
SYNC	1 byte	`0x55`
R/W	1 bit	1 write, 0 read
#words	7 bits	word count, 1 to 127
Base address	4 bytes	byte address, 4 byte aligned
Write data	4 bytes each	#words words, write only

Table 11: Request packet

Field	Size	Wrapper to PC
SYNC	1 byte	`0xAA`
valid	1 bit	1 accepted, 0 refused
#words	7 bits	echoed word count
Base address	4 bytes	echoed base address
Read data	4 bytes each	#words words, read only

Table 12: Response packet

Points worth knowing about the framing:

- Every field wider than a byte goes **MSB first**, both the base address and the data words. The header is six bytes, the payload 4× #words bytes.
- The R/W bit and the word count **share one byte**: bit 7 is R/W, bits 6 downto 0 are the count. The response byte is the same shape, with *valid* in bit 7.
- **One packet moves at most 127 words.** Longer transfers are split across packets with the base address advancing by four per word. This is routine and not an edge case: reading the stock 2048-sample buffer at stride 4 is 8192 words, that is 65 packets.
- **A write is acknowledged too.** The response header goes out as soon as the address has been parsed, before the write data has even arrived, so a host that does not drain the six byte acknowledgement will desynchronise the next transaction. Only a read is followed by data words.
- Because the write header precedes the write payload, an overflow during that payload is reported on the *next* packet's header. Overflow detection is best effort: valid = 0 means bytes were dropped, valid = 1 does not promise that none were.

10 Additional references

- **Repository and full documentation** — github.com/yashparitkar/libreila. This datasheet is the specification for the register map, the trigger vector and the packet format; the repository holds the sources, the drivers, the test suite and the per-directory READMEs covering code generation and driver usage.
- **LibreILA User Manual** — the full instantiation, generation and debug procedure, of which this datasheet is a summary.
- **License** — CERN Open Hardware Licence (OHL) Version 2, Permissive. See the `LICENSE` file in the repository.
- **Microchip G238606** — PolarFire family fabric user guide, for the LSRAM block and its independent read, which the readout path relies on.
- **ARM IHI 0022** — AMBA AXI and ACE protocol specification, for the AXI4Lite slave port.
- **VHDL Style Guide (VSG)** — recommended for reformatting the generated sources before they are committed to a project.

11 Device utilization and performance

Resource usage scales with the two generics that describe the capture, and with nothing else. The storage requirement is exactly

$$\text{bits} = \texttt{G\_SAMP\_BUFF\_DEPTH} \times \texttt{C\_N\_LANES} \times 32$$

that is, the probe word rounded up to a whole number of 32-bit lanes. Note that the stride padding used on the bus costs address space only and is not stored, so a 67-bit probe occupies three lanes in RAM and not four. For the stock build that is $2048 \times 3 \times 32 = 196{,}608$ bits, which maps onto PolarFire LSRAM blocks.

This design uses UART at 115200 baud and size of both the transmit and receive FIFOs is 1024 bytes. The core is clocked at 100 MHz for both clock domains.

Probe width	Depth	Fabric 4LUT	Fabric DFF	Interface 4LUT	Interface DFF	LSRAM (20K)	fmax
67	2048	1280	817	360	360	9	243 MHz
67 (ext_trig)	2048	1015	558	360	360	9	270 MHz
67	4096	1267	828	540	540	14	252 MHz
35	2048	1238	817	288	288	7	252 MHz

Table 13: Device utilization, PolarFire SoC. Includes the UART wrapper (UART, TX/RX FIFOs and packet parser) around the `libre_ila` core.

Table 14 isolates the `libre_ila` core itself, that is the sample buffer, trigger comparator, address decode and AXI4Lite slave state machine, excluding the UART wrapper that surrounds it in the IP used for this measurement.

Probe width	Depth	Fabric 4LUT	Fabric DFF	Interface 4LUT	Interface DFF	LSRAM (20K)
67	2048	600	388	288	288	7
67 (ext_trig)	2048	352	129	288	288	7
67	4096	595	395	468	468	12
35	2048	556	388	216	216	5

Table 14: Device utilization of the `libre_ila` core module alone, PolarFire SoC.

Generic	Default	Range	Description
G_SAMP_CLK_FREQ	100 000 000	> 0	Sampling clock frequency in Hz. Reported over AXI4Lite so the host can build the time axis; it configures no logic.
G_AXIL_CLK_FREQ	100 000 000	> 0	AXI4Lite clock frequency in Hz. Sets the wrapper's watchdog period and the UART divider, so it must match the real clock.
G_EXTERNAL_TRIG	0	0 or 1	1 uses the i_ext_trig pin instead of the trigger vector.
G_PROBE_WIDTH	67	≥ 1	Total probed bits. Keep it a multiple of 32 for best results. Checked at elaboration. Bounded above only by available fabric RAM.
G_SAMP_BUFF_DEPTH	2048	$2^k, k \geq 1$	Number of samples stored. Must be a power of two greater than one, checked at elaboration. Bounded above only by available fabric RAM.
G_UID	0	0x0–0x7FFFFFFF	Identity of this instance, reported at UID so that a system holding several cores can tell them apart. Configures no logic and every value is legal, zero meaning unset.
G_UART_RX_FIFO_DEPTH	1024	2^k	Wrapper only. Depth of the UART receive FIFO in bytes. Must be a power of two, which is not checked at elaboration.
G_UART_TX_FIFO_DEPTH	1024	2^k	Wrapper only. Depth of the UART transmit FIFO in bytes. Must be a power of two, which is not checked at elaboration.
G_BAUD_RATE	115 200	$\leq$ G_AXIL_CLK_FREQ/32	Wrapper only. UART baud rate, 8N1. The bound gives the 16× oversampling receiver two clocks per sample and is checked at elaboration.
C_S_AXIL_DATA_WIDTH	32	32	AXI4Lite data width. Do not change.
C_S_AXIL_ADDR_WIDTH	32	32	AXI4Lite address width. Do not change.

Table 7: Configuration generics

Type	Slave side	Master side	Pass through
in	input	output	yes, slave to master
out	output	input	yes, master to slave
mon	input	none	no, parallel tap only

Table 8: Probe signal types in portmap.csv

Port	Dir	Width	Description
i_rst_sync	in	1	Synchronous reset, active high
samp_aclk	in	1	Sampling clock, the clock of the probed domain
i_ext_trig	in	1	External trigger input, rising edge sensitive, used when G_EXTERNAL_TRIG is 1
o_trig_out	out	1	Trigger output, for chaining to further ILA instances
s_axil_aclk	in	1	AXI4Lite clock
s_axil_aresetn	in	1	Core only. AXI4Lite reset, active low; the wrapper drives it internally
s_axil_*	—	—	Core only. AXI4Lite slave channels, C_S_AXIL_DATA_WIDTH wide
uart_rx	in	1	Wrapper only. Serial receive
uart_tx	out	1	Wrapper only. Serial transmit
probe_slave_*	—	—	Probe port facing the master of the probed link
probe_master_*	—	—	Probe port facing the slave of the probed link, every direction mirrored

Table 9: Fixed interface signals